Indicode Compiler Front-End Status Report

Status date: 2026-11-03

Scope: Indicode front end only; LLVM/backend/runtime work is excluded.

The companion catalogue audit PDF
contains the full 40-item checklist and remaining-work detail; it is generated
from FRONT_END_REMAINING_WORK.md.

Executive summary

The front end reads Indicode source, tokenizes and parses the supported
language, checks names and types, reports source-aware diagnostics, and lowers
valid input to a typed HIR that is independently verified.

Measure	Current status
Weighted front-end completion estimate	97.10% (rounded to 97%)
Front-end catalogue items done to current v1 scope	30 / 40
Partially implemented catalogue items	0 / 40
Not started catalogue items	10 / 40
Current CTest suite	19 / 19 passed

The 97.10% figure is a judgment-based, weighted estimate of the front-end
acceptance areas below. It is not the percentage of the 40-item catalogue:
30/40 items equals 75% by count, while the weighted estimate gives much more
credit to mature build, lexer, parser, and semantic areas. Neither figure
means that the entire language, LLVM compiler, runtime, or original 777-item
project catalogue is complete.

Scope and limitations

The agreed working boundary is a documented, single-file Indicode v1 front
end that can produce verified typed HIR. Grammar and semantics v1 are
specified in LANGUAGE_SPEC_V1.md and
SEMANTICS_V1.md.
Modules/imports, overloads, generic declarations and their constraint/
substitution engines, ownership/borrowing, effect checking, symbol indexing,
and incremental parsing are not implemented. Several are explicitly outside
the current v1 syntax/semantic contract; they remain visible in the catalogue
as future work rather than being claimed as completed. There is no LLVM
adapter, machine-code generation, executable runtime, tensor execution, or
device backend. The project is not production-ready: full conformance,
coverage-guided fuzzing, sanitizer gates, release CI, and compatibility/ABI
policy remain.

Weighted progress by acceptance area

Area	Weight	Done	Points	Evidence and remaining gap
Project/build foundation	5%	95%	4.75	C++20/CMake library, CLI, CTest, install/export package and separate package consumer; release CI remains.
Source manager and diagnostics	10%	97%	9.70	Owned source buffers, indexed line tables, byte/scalar columns, tab-aware diagnostics, notes, fix-its and JSON schema v1; terminal display-width and editor integration remain.
Lexer	15%	92%	13.80	Typed token payloads, trivia, UTF-8 validation, strings, radix integers, separators and nested comments; Unicode identifiers, suffix policy and complete numeric grammar remain.
Parser and AST	15%	99%	14.85	Grammar v1, precedence, recovery, golden printer, exhaustive visitor and ownership/lifetime contract; broader conformance remains.
Name resolution	10%	95%	9.50	v1 scopes/namespaces, shadowing, stable identities and HIR binding checks; modules/imports/overloads remain future work.
Type checking and semantic validation	15%	99%	14.85	v1 declarations, expressions, calls, conversions, records, collections, tensor shapes, constants and conservative flow/return checking; ownership/effects and tensor operations remain outside v1.
Typed HIR and verification	15%	99%	14.85	AST lowering, stable symbols, schema-v1 JSON and independent checks of types, scopes, identities, calls, mutability and return paths; deserialization and broader compatibility guarantees remain.
CLI and diagnostics tooling	5%	98%	4.90	**check**, **tokens**, **ast**, **ast-json**, **typed**, **typed-json**, source excerpts and schema-v1 diagnostics; editor API and terminal display-width support remain.
Tests, docs and integration	10%	99%	9.90	19 passing tests including malformed-input smoke tests and installed-package consumer; full conformance, fuzzing, sanitizers and CI remain.
Total	100%		97.10%	Rounded display: 97%.

Calculation: **4.75 + 9.70 + 13.80 + 14.85 + 9.50 + 14.85 + 14.85 + 4.90 + 9.90 = 97.10**.

What is implemented

Source model and lexer

- Owned source files, stable source IDs, spans, indexed CR/LF/CRLF line lookup, byte offsets, UTF-8 scalar columns, and four-column tab stops.
- Stable lexer diagnostics and recovery for malformed tokens, UTF-8, numeric forms, strings, and comments.
- Typed keyword/integer/float/string payloads; original spelling and source spans are retained.
- Source-spanned whitespace and line/block-comment trivia; nested block comments preserve newline separators.
- ASCII identifiers, supported string escapes and Unicode scalar escapes, decimal floats, binary/octal/hex integers, and digit separators.

Grammar, parser and AST

- Normative grammar version 1.0.0, recursive-descent declarations/statements, and Pratt expression parsing with documented precedence/associativity.
- Functions, records, typed fields, lists, calls with named arguments, indexing, member access, loops, integer ranges, conditionals and nested **else if**.
- Declaration, statement and delimited-expression recovery; valid later statements and sibling expressions are retained after malformed input.
- Deterministic AST printer golden output; public **AstVisitor** callbacks for every declaration, statement kind and expression kind; full-tree **walk_ast**.
- Documented AST ownership/lifetime contract. **Program** owns syntax nodes; nodes do not borrow token/source text. Source contents must still be retained separately when source excerpts are needed.

- Schema-versioned AST JSON and source spans.

Name binding, types and semantic checks

- Separate v1 value/function/type lookup rules, lexical block scopes, permitted outer-value shadowing, duplicate declaration diagnostics, and stable identities for values, functions, record types/fields, builtins and references.
- Type inference/checking for supported declarations, literals, expressions, operators, function calls, named arguments, records, lists, text, indexing, integer ranges, and checked numeric conversions.
- Static tensor type validation for numeric elements and exact rank, dimensions, and element types. Symbolic dimensions match by name; tensor operations/broadcasting/device placement are not supported.
- Bounded integer/Boolean constant evaluation with short-circuiting, signed-64-bit overflow and division-by-zero checks, and a deterministic 256-node evaluation budget.
- Condition/count/return checks, unreachable-statement diagnostics, and conservative definite-return analysis. A loop is not assumed to execute.
- Semantic rules are tested and independently rechecked in typed HIR where applicable.

Diagnostics, HIR, CLI and packaging

- Text diagnostics with source excerpts/carets, stable severity/code identifiers, notes/secondary spans, fix-its and multi-error reporting.
- Diagnostic JSON schema v1 with version envelope, locations, notes, fix-it data, escaping, deterministic ordering and golden-output tests.
- Project-owned typed HIR lowering and independent verification of types, symbols, visibility, mutability, calls, loops, returns and control flow.
- CLI commands: **check**, **tokens**, **ast**, **ast-json**, **typed**, and **typed-json**; JSON output is schema-versioned.
- CMake install/export package and a separately built package-consumer test.

Catalogue audit: all 40 front-end items

The original front-end catalogue covers IDs 445–484. “Done” means accepted for the documented, implemented v1 scope; it does not imply future features listed under “Not started”.

Status	IDs	Concepts
Done (30)	445–459	Source manager, source buffer, source span, line table, column mapping, UTF-8 validator, token kind, token payload, lexer state, trivia preservation, parser state, recursive-descent parser, Pratt parser, grammar version, AST context.
Done (30)	461–467	AST visitor, AST printer, AST serializer, syntax recovery, error synchronization, symbol identity, scope table.
Done (30)	471–472	Name binding, type checker.
Done (30)	475–477	Constant evaluator, control-flow validation, definite-return analysis.
Done (30)	480–482	Tensor shape checker, diagnostic engine, JSON diagnostics.
Not started (10)	460	AST node arena / ownership alternative measurement. The current AST uses ordinary C++ value/**unique_ptr** ownership.

Status	IDs	Concepts
Not started (10)	468–470	Module resolver, import resolver, and overload candidate selection.
Not started (10)	473–474	Generic constraint solver and type substitution engine.
Not started (10)	478–479	Effect checker and ownership/borrowing checker.
Not started (10)	483–484	Language-server symbol/reference index and incremental parsing.

There are no catalogue items currently marked “Partial”. The ten not-started items are not hidden by the 97% weighted estimate.

Remaining work to reach the next milestones

1. AST scalability: measure large-input allocation/destruction and decide whether an arena is beneficial or whether documented **unique_ptr** ownership remains the chosen design (ID 460).
2. Modules and imports: define module identity, search roots, import aliases, visibility, missing-import/ambiguity diagnostics and cycle behavior (IDs 468–469). This requires a language-scope decision beyond single-file v1.
3. Overloads and generics: decide whether overloads/generics enter a future grammar version, then implement overload selection, constraint solving and type substitution with ambiguity and failure tests (IDs 470, 473–474).
4. Effects and ownership: specify effect and ownership/borrowing models before implementing their checkers; these cannot be safely inferred from the current v1 rules (IDs 478–479).
5. Editor and incremental support: build a source-aware definitions/references index, define edit invalidation, and verify incremental parses match clean parses (IDs 483–484).
6. Production/release readiness: add requirement-linked full v1 conformance, coverage-guided fuzzing, sanitizers, clean offline CI/release builds, and a documented compatibility policy. These are acceptance gaps reflected in the weighted areas even when not separate catalogue IDs.

Verification recorded for this report

- CMake build succeeds in the out-of-tree directory **/tmp/indicode-partial-build**.
- Full CTest suite passes: 19/19.
- Covered tests include lexer/parser/AST/visitor/semantics/HIR/diagnostics, CLI JSON, malformed-input robustness, and installed-package consumption.
- **git diff --check** passes for the report update.

Related specifications

- Language syntax v1
- Semantic rules v1
- AST ownership and lifetime
- Parser recovery v1
- Diagnostic schema v1
- Detailed catalogue checklist

- Detailed catalogue checklist PDF